

# Error Handling

## HTTP Status Codes vs Business Error Codes

GameHub uses a **two-layer error model**:

- **HTTP status codes** reflect **transport/infrastructure-level** outcomes — was the request delivered at all?
- `result_code` in the response body reflects **everything else** — authentication, business logic, validation.

### Integrator parsing rule

**One simple rule:** if you receive **HTTP 200**, parse `result_code` from the body. That's it.

Non-200 responses are infrastructure-level problems where the request never reached business logic — no `result_code` to parse.

| HTTP Status | Meaning                                                                     | Body                                                                                               |
|-------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 200         | Request received and processed. Check <code>result_code</code> for outcome. | Always: <code>result_code</code> , <code>message</code> (if error), plus endpoint-specific fields. |
| 400         | Malformed request — server could not parse the JSON.                        | <code>message</code> only. No error codes.                                                         |
| 403         | IP not whitelisted — request blocked at infrastructure level.               | <code>message</code> only. No error codes.                                                         |
| 500         | Internal server error.                                                      | <code>message</code> only. No error codes.                                                         |

**Important: All business logic goes through HTTP 200**

All wallet operations (**bet, win, rollback, balance**) return **HTTP 200** for every scenario where the server understood the request – including authentication failures (wrong credentials, expired token), validation errors (invalid game, invalid currency), and business rejections (insufficient balance, bet not found). The `result_code` field tells the integrator exactly what happened. Non-200 HTTP codes are reserved for infrastructure failures only (unparseable request, IP blocked, server crash).

## Error Response

Infrastructure errors (HTTP 400 / 403 / 5xx) return **no response body**. The HTTP status code itself is sufficient – these are not actionable by the integrator programmatically.

Business error structure (HTTP 200 with non-zero `result_code`):

```
{
  "result_code": 1012,
  "message": "Insufficient balance",
  "balance": 5.00,
  "currency": "USD"
}
```



## Error Codes

### Business Error Codes (returned with HTTP 200)

All error codes below are returned inside an **HTTP 200** response via the `result_code` field.

| Code | Name / Description | Typical Cause                                                                         | What to do on Integrator Side                                                                      |
|------|--------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 1001 | Invalid signature  | X-Sign verification failed (wrong key, wrong field order, amount formatting mismatch) | Verify signature calculation matches the <a href="#">X-Sign spec</a> . Check amount normalization. |
| 1002 | Invalid            | Wrong Operator ID or                                                                  | Verify                                                                                             |

|      | credentials          | key                                                                    | credentials/config,<br>rotate keys if needed.                       |
|------|----------------------|------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1003 | Invalid game         | Unknown or disabled game ID                                            | Do not retry; update game catalog / configuration.                  |
| 1004 | Invalid player       | Player not found or blocked                                            | Stop operation; show error to user; check KYC/limits.               |
| 1005 | Invalid currency     | Unsupported currency                                                   | Map only allowed currencies; correct config.                        |
| 1006 | Invalid session      | Session expired or unknown                                             | Force re-login / re-launch game; obtain new session.                |
| 1007 | Invalid token        | Auth token expired or corrupted                                        | Refresh token or start new session; do not reuse old token.         |
| 1008 | Invalid round        | Non-existing or already closed round                                   | Stop retries; review game flow / idempotency.                       |
| 1009 | Invalid bonus        | Bonus program/instance invalid                                         | Do not retry; synchronize bonus configuration.                      |
| 1011 | Bet not found        | <code>parent_transaction_id</code> does not exist in operator's system | Do not credit/refund; log for investigation; verify bet was sent.   |
| 1012 | Insufficient balance | Bet amount exceeds player's available balance                          | Show "insufficient funds" to player; do not retry with same amount. |
| 1013 | Loss limit           | Player's loss limit                                                    | Stop gameplay; show                                                 |

|      |                          |                                                                                   |                                                                          |
|------|--------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------|
|      | exceeded                 | reached for the configured period                                                 | responsible gaming message to player.                                    |
| 1014 | Wager limit exceeded     | Player's wagering limit reached for the configured period                         | Stop gameplay; show responsible gaming message to player.                |
| 1015 | Time limit exceeded      | Player's session time limit reached                                               | End session; show responsible gaming message to player.                  |
| 1016 | General limit exceeded   | Other responsible gaming limit (e.g. deposit limit, reality check)                | Stop operation; show responsible gaming message to player.               |
| 1017 | Bet already settled      | Rollback references a bet that already has a corresponding win or round is closed | Do not refund; bet remains in GGR.                                       |
| 1100 | Could not start the game | Launch error on aggregator/operator side                                          | Show generic error to user; allow re-try; contact support if persistent. |

## Infrastructure Errors (HTTP non-200)

These indicate the request never reached business logic. No `result_code` is returned — only a `message`.

| HTTP Status | When                                    | What to do on Integrator Side                    |
|-------------|-----------------------------------------|--------------------------------------------------|
| 400         | Malformed JSON — server could not parse | Fix the request payload; this is a client bug.   |
| 403         | IP not on whitelist                     | Contact integration manager to update whitelist. |

|     |                                  |                                                              |
|-----|----------------------------------|--------------------------------------------------------------|
| 500 | Unhandled server error           | Retry with exponential backoff; if frequent, notify support. |
| 502 | Upstream unavailable             | Retry with backoff; fail gracefully in UI.                   |
| 503 | Temporary overload / maintenance | Retry after delay; optionally show maintenance message.      |



#### Retry Guideline

- **HTTP 200 with non-zero `result_code`** — do **not** retry. The server processed the request and rejected it.
- **HTTP 400 / 403** — do **not** retry. Fix the request or infrastructure first.
- **HTTP 5xx** — **retry** with exponential backoff (max 3–5 attempts). These are transient failures.

## GameHub Retry Policy (calls to your wallet)

The above section is about your retry behaviour when **calling GameHub**. This section is the inverse — what **GameHub does when calling your wallet endpoints** ( `/seamless/bet` , `/seamless/win` , `/seamless/rollback` ). The distinction matters for designing your idempotency and rate-limit behaviour.

### How GameHub classifies your response

| Your response                                      | GameHub treats it as                                    | What happens to the bet                                                                                                             |
|----------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>HTTP 200 +</b><br><code>result_code == 0</code> | Success                                                 | The operation is confirmed (bet placed / win settled / rollback applied). No further calls.                                         |
| <b>HTTP 200 +</b><br><code>result_code != 0</code> | <b>Terminal reject</b> — your definitive business "no". | <b>GameHub will NOT retry.</b> The rejection is recorded as the final outcome of the call; the <code>result_code</code> is kept for |

|                                                              |                                                                                                                                       |                                                       |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
|                                                              |                                                                                                                                       | audit/support.                                        |
| <b>HTTP 4xx</b>                                              | Retryable. Treated as a contract violation on the calling side (us) or a transient operator-side block (auth, signature, rate-limit). | GameHub retries with exponential backoff (see below). |
| <b>HTTP 5xx, timeout, connection refused, DNS failure</b>    | Retryable. Standard transient operator-side incident.                                                                                 | Same retry ladder.                                    |
| <b>Malformed / unparseable body, missing required fields</b> | Retryable.                                                                                                                            | Same retry ladder.                                    |



#### HTTP 4xx is NOT a way to terminally reject a bet

If you intend to reject a bet, return **HTTP 200 with** `result_code != 0` — that is the contracted business-reject channel and the only path that produces a single-attempt terminal rejection. Returning HTTP 4xx (e.g. 422 from a validation framework, 401 from a signature failure) will be retried by GameHub through the full retry ladder before the bet is given up on. This is deliberate: HTTP 4xx is too broad a category to silently terminal-reject a bet on — auth bugs, signature mismatches, WAF rate-limits, and config drift all live there.

## Retry ladder

GameHub uses exponential backoff: `delay = baseDelay × 2^attempt`, capped at 24h. With the production configuration (`baseDelay = 5s`, `maxRetries = 15`) the schedule is:

| Attempt # | Delay since previous | Elapsed since first failure |
|-----------|----------------------|-----------------------------|
|           |                      |                             |

|                  |              |                  |
|------------------|--------------|------------------|
| 1 (initial call) | —            | 0s               |
| 2                | 5s           | 5s               |
| 3                | 10s          | 15s              |
| 4                | 20s          | 35s              |
| 5                | 40s          | 1 min 15s        |
| 6                | 1 min 20s    | ~3 min           |
| 7                | 2 min 40s    | ~5 min           |
| 8                | ~5 min       | ~11 min          |
| 9                | ~11 min      | ~21 min          |
| 10               | ~21 min      | ~43 min          |
| 11               | ~43 min      | ~1 h 25 min      |
| 12               | ~1 h 25 min  | ~2 h 50 min      |
| 13               | ~2 h 50 min  | ~5 h 40 min      |
| 14               | ~5 h 40 min  | ~11 h 20 min     |
| 15               | ~11 h 20 min | ~22 h 40 min     |
| 16 (final)       | ~22 h 40 min | <b>~45 hours</b> |

If all 16 attempts fail, GameHub stops calling for that `transaction_id`. The bet is reconciled internally on the GameHub side; you may safely assume **no further calls will be made for that `transaction_id`** unless one of your operators contacts support and asks for a manual replay.

## What you need to support

1. **Idempotency on `transaction_id`** — GameHub will resend the **same `transaction_id`** on every retry. Your wallet **MUST** treat a repeat with the same `transaction_id` as the same logical operation, returning the original result (do not double-charge / double-credit).
2. **Stable response shape across retries** — if attempt 3 succeeds, the response shape (balance, transaction\_id, result\_code=0) must match what you would have returned on attempt 1. Don't return different shapes mid-retry.
3. **Don't rely on a single-shot delivery** — your wallet should be designed assuming GameHub may call the same endpoint many times for the same `transaction_id` over the course of ~45 hours, especially during operator-side incidents.